

Tyler Gilbert, Ashton Hull, Jacob Lehtola, Braxton Wilson
Dr. John Nicholson
CSCI 4805
May 06 2026

Pantry Chef Final Report

Introduction

Everyone needs to eat, and in order to eat, food needs to be cooked. Some of the biggest barriers to entry in cooking are not knowing what to cook with the ingredients you have on hand and difficulty keeping track of which ingredients you do or do not have. This causes people to either avoid cooking altogether or buy multiple of the same ingredients. Pantry Chef is an application that serves not only as a mobile pantry for storing the user's ingredients, but also as a recipe recommender based on those ingredients. The app aims to make cooking more easily accessible to everyone and help reduce wasteful spending on duplicate ingredients.

The primary purpose of this project is to simplify meal decision-making by organizing food items and suggesting meals that can be made using only existing ingredients. The app makes adding ingredients easy by allowing users to scan food item barcodes to add them quickly. Once the ingredients are in the pantry, users can search for recipes they can currently make. Pantry Chef encourages users to make sustainable food choices and helps them discover new recipes and cuisines without extra spending. The app also supports people starting their cooking journey with detailed instructions. In addition, once the user finishes a recipe, the app will automatically deduct the ingredients from the user's pantry. By emphasizing clarity and real-world usability, the project serves a wide audience. Pantry Chef provides a practical solution that encourages resource awareness, reduces reliance on dining out, and supports cooking habits that are both approachable and effective.

Technology

The technologies used in Pantry Chef had a learning curve, especially React Native. React Native was challenging at first because Tyler was the only team member who had prior experience with it. The rest of the team had to learn how mobile components, styling, navigation, and state management worked while also contributing to the project. This made the beginning of development slower because the team was not only building the app, but also learning how mobile development works compared to traditional web or desktop development. Even though this took time, the team was able to adjust and become more comfortable with React Native as development continued.

React Native was chosen because it allowed the team to build Pantry Chef as a cross-platform mobile application. Instead of creating separate projects for Android and iOS, React Native allowed most of the app to be written using one shared codebase. This was useful because Pantry Chef needed to feel like a real mobile app with multiple screens, forms, navigation, barcode scanning, and account features. React Native also made it easier to create reusable components, such as ingredient cards, recipe cards, search bars, buttons, and form

screens. Once these components were built, they could be reused throughout the app, which helped keep the design consistent.

Expo was also an important part of the project. Expo made development easier because it allowed the team to test the app quickly on emulators and physical devices. Without Expo, setting up and testing a mobile app would have been more complicated, especially for team members who were newer to mobile development. Expo also helped with features like barcode scanning and running the app through Expo Go. This made it easier to see changes in real time and test the application without building a full app package every time. Expo still required some setup and troubleshooting, but overall it made the development process smoother.

TypeScript was used to make the project more reliable and easier to maintain. Since Pantry Chef uses many different screens, services, database tables, and API responses, TypeScript helped catch mistakes while writing code. This was useful when passing data between screens, updating pantry ingredients, saving recipes, and working with Supabase database types. TypeScript sometimes made development slower because the team had to fix type errors before continuing, but this was helpful in the long run. It reduced the number of runtime errors and made the code easier to understand as the project became larger.

Supabase was chosen as the backend technology for Pantry Chef. It provided authentication, database storage, and security features through row-level security policies. Supabase was used to store important app data such as users, pantries, pantry ingredients, user preferences, and saved or made recipes. This made Pantry Chef more than just a front-end project because user data could persist after the app was closed. Supabase was one of the more difficult technologies to learn because the team had to understand tables, migrations, relationships, authentication, and security rules. However, it made development easier once the structure was in place because the app could use Supabase methods to read and update data without writing a separate backend from scratch.

The Spoonacular API was used for ingredient and recipe data. This API allowed Pantry Chef to look up food items, retrieve ingredient images, search for recipes based on pantry ingredients, get recipe instructions, and support barcode-related features. Spoonacular was chosen because creating a full recipe and ingredient database ourselves would have been outside the scope of the semester. It gave the app access to real recipe data and made the recipe recommendation feature possible. However, Spoonacular also created some challenges. The API was rate-limited, so the team had to be careful about making too many requests. In addition, recipe matching was not always predictable, which made it harder to guarantee that the app would always return the exact recipe results expected.

TanStack Query, also known as React Query, was used to manage cached API data. This was important because Pantry Chef needed to avoid making repeated Spoonacular requests when the same information had already been loaded. Caching helped reduce unnecessary network calls and made the app feel faster. The team configured cached API responses with a one-hour expiration and disabled automatic refetching to avoid using too many API calls. TanStack Query had a learning curve because the team had to understand query clients, cache time, stale time, and providers. However, it was the correct choice because it helped control API usage and improved the app's performance.

React Navigation was used to organize the app into different screens. Pantry Chef includes authentication screens, onboarding screens, the pantry page, recipe browser, barcode scanner, recipe detail screen, and account page. React Navigation made it possible to separate these features into their own screens while still allowing users to move through the app naturally.

It also helped control whether a user should see the sign-in flow or the main app flow depending on their login status. React Navigation was not the most difficult technology to learn, but it did require practice because the team had to understand navigators, screen stacks, route parameters, and how to pass data between screens.

Expo Camera was used for the barcode scanner. This feature allowed the app to access the phone's camera and scan UPC barcodes so users could add pantry ingredients more quickly. This made Pantry Chef more practical because users do not have to manually type every ingredient into the pantry. Expo Camera was somewhat challenging because the team had to work with camera permissions, barcode types, scanner layout, and error handling. Even though it took extra work, it was worth including because barcode scanning became one of the most important features of the app.

Lottie was used for the onboarding animations. These animations helped explain the purpose of the app in a more visual and engaging way. The onboarding section uses animations related to food waste and saving money, which connects directly to Pantry Chef's main goals. Lottie was a better choice than other formats, such as GIF, because it's a vector format. Lottie files are resolution independent, making them a great choice for cross platform apps with unpredictable screen resolutions. It helped make the app feel more polished without requiring the team to create complex animations manually in React Native.

Overall, the technologies made development somewhat more difficult in the beginning, but they did not cause any significant delays. Most of the challenge came from learning how each technology connected to the others. For example, the team had to learn how the React Native screens communicated with service functions, how those service functions communicated with Supabase or Spoonacular, and how cached data affected the app's behavior. Once the team understood the overall structure, development became easier and more organized.

Originally, the team discussed using native Android development to build the app. However, we decided to use React Native instead because it allowed us to create a cross-platform mobile app and present a more impressive project at the Innovation Experience. Native Android development would have worked for building an Android app, but it would have limited the project more heavily to one platform and required a different development approach. React Native was the better choice because it allowed the team to build a modern app with reusable components and the possibility of running on both Android and iOS.

This was the correct decision because React Native helped Pantry Chef become a stronger final product. Although it was more difficult to learn at first, it gave the app a professional mobile structure and allowed the team to include features like navigation, barcode scanning, authentication, database storage, recipe browsing, and onboarding animations. The technologies required the team to learn many new skills, but they also helped turn Pantry Chef into a more complete and realistic software project.

Design

Pantry Chef uses a minimal design with a navigation bar and four pages. The app uses a white-and-orange color scheme across all screens. This consistency helps the app achieve a cohesive look.

Overview



Figure 1. Log in screen for Pantry Chef

When first starting the app, the user will be prompted to sign in. Currently, only Google emails are usable for sign-in (Figure 1). If the user has an account, the user can sign in using a valid Google email and password stored in the user database. If the user has never logged in, they will be prompted to complete the entire onboarding process. The user will be prompted to register an email address, a password, and a name that will be displayed when the app loads. Once the user opens the app, it will be split into four screens. The four main screens of the app consist of:

1. Pantry page: users can store their ingredients and edit ingredient information.
2. Recipe browser: once opened, queues all ingredients in the user's pantry and lists all recipes that can be made with them.
3. Barcode scanner: allows the user to scan food bar codes in order to quickly add ingredients to the user's pantry
4. Account page: displays the user's profile settings, including changing the email and password, and allows the user to go through the onboarding process again if they choose.

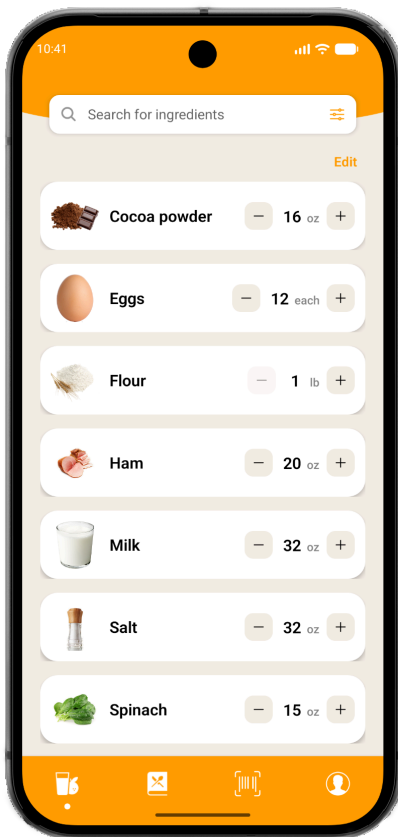
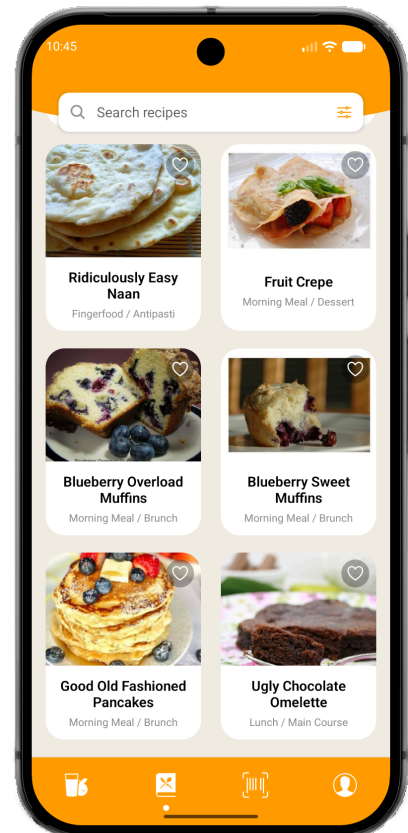


Figure 2. Pantry Page for Pantry Chef

Once the user opens the pantry page, if the user is new, no ingredients will be listed; however, once ingredients are added to the pantry, they will be shown in these ingredient cards. The cards are default sorted by alphabetical order and display the name of the ingredient, an image for the ingredient which comes from the Spoonacular API, and each ingredient has a number of ounces, which can either be manually adjusted or the user can click on the number, and it can be edited that way. Once the user clicks an ingredient card, the edit ingredient screen will pop up, allowing them to update all ingredient information, including the name, quantity, food category, and expiration date. Even though the user can change all information about an ingredient, the app will still see the ingredient as the original one added to the pantry, since the API assigns each ingredient an ID that the user cannot modify. The ingredient page has a search bar at the top to search the pantry, and a filter to list ingredients in reverse alphabetical order.

Figure 3. Recipe browser for Pantry Chef

Once the user opens the recipe browser page, the ingredients from the pantry will be retrieved via a function, and the page will auto-populate with recipes that can be made with those ingredients. Once the user clicks the recipe, the recipe page opens and displays information about the recipe, including estimated cooking time, a list of ingredients, and the recipe's instructions. At the end of the recipe, there is a button to finish it, and once the user presses it, the ingredients will be automatically deducted from the user's pantry. The user can save recipes by clicking the heart icon in the top right corner of each recipe card, so even if they do not have the ingredients in the pantry, they can save them. There is also a history feature so that once a recipe is made, the data will be saved even after the user finishes cooking.



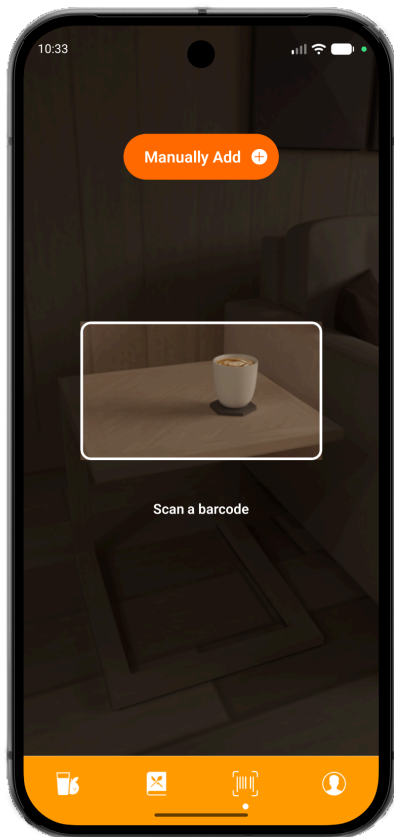
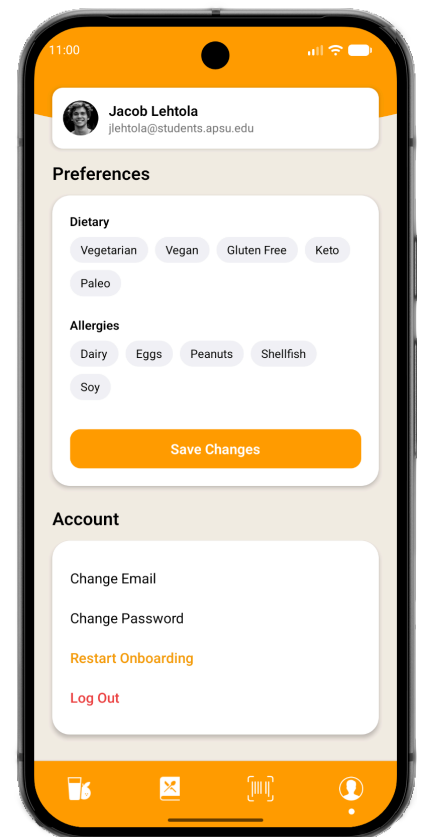


Figure 4. Barcode Scanner for Pantry Chef

The barcode scanner is one of the most important features in the Pantry Chef app. The barcode scanner uses the phone's camera in order to scan UPC barcodes. Once the barcode is successfully scanned, the same editing-ingredients page in the pantry will be displayed. Once the page is displayed, the name will auto-populate, and the user can give the ingredient a different name, since the default name is sometimes too long. The user has to manually add the ingredient quantity, since just scanning a barcode does not indicate whether the ingredient is full, the ingredient type, or the expiration date (optional). Once the ingredient is scanned and everything on the page is correct, the user can click the add button, and the ingredient will automatically be added to the pantry. If the barcode is damaged or does not scan, the user can press the "Manually Add" button to open the same page, but the name will not auto-populate.

Figure 5. Profile Screen for Pantry Chef

The final screen the user can access in the app is the profile page. The profile page displays information about the user, including their username, email address, dietary restrictions, and allergies. The page also allows the user to change the email and password (only the email reset currently works), restart the onboarding process, and log out of the account.



Onboarding

The onboarding sequence uses Lottie animations to visually convey the app's purpose and foster a premium feel. It presents relevant facts about food waste and saving to clearly position the app and what it does. The animations were created using [Lottie Lab](#) and implemented inside the app using the lottie-react-native module.



Figure 6. Lottie animation with moving trash items and a trash can.

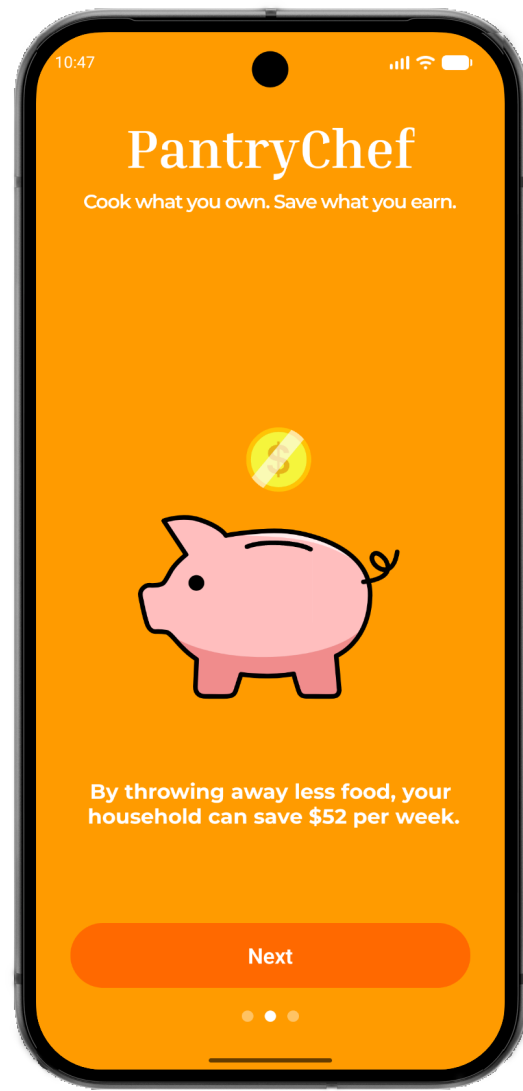


Figure 7. Lottie animation with a moving coin and a piggy bank.

Database

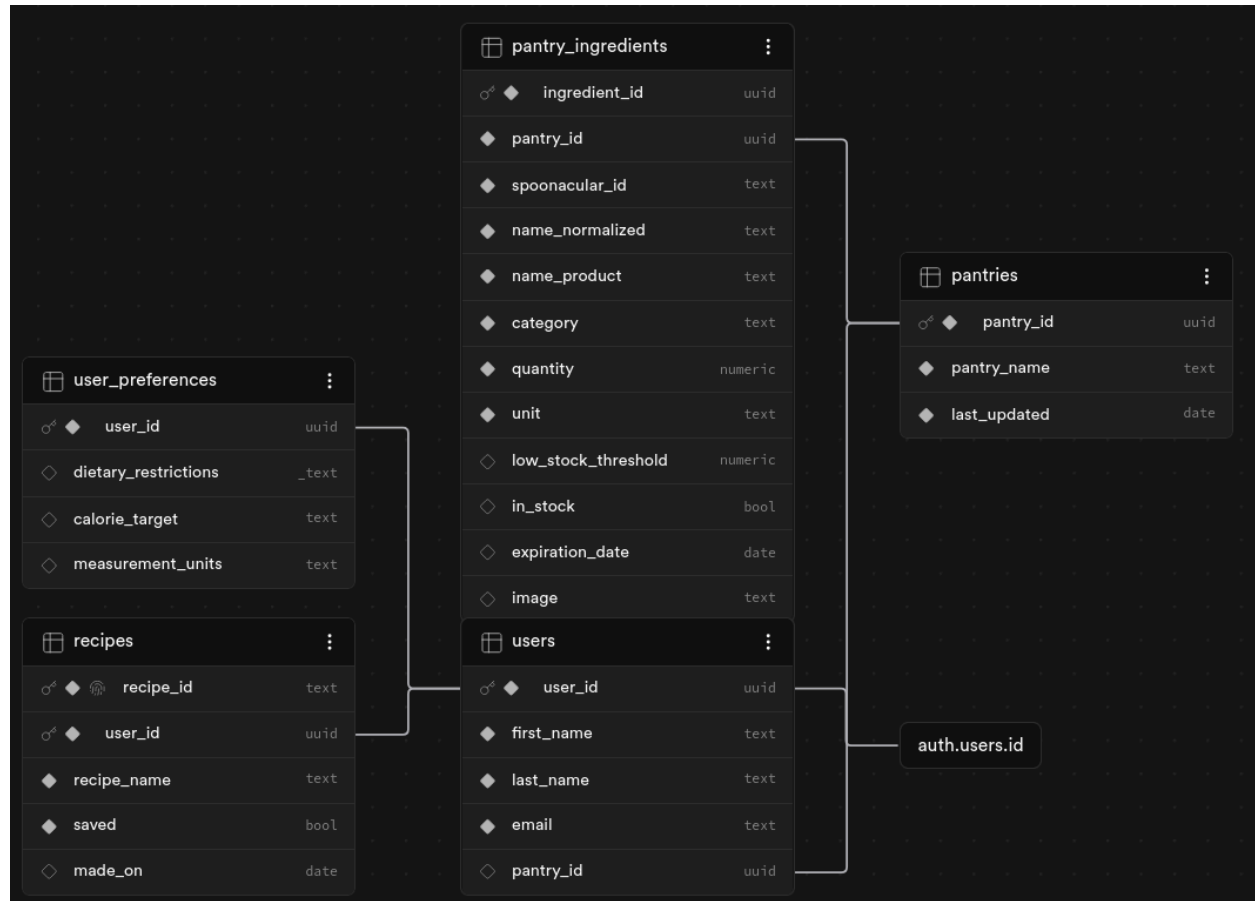


Figure 8. Database entity-relation diagram.

Pantry Chef uses Supabase as its backend, which provides a hosted PostgreSQL database and a template for writing queries within defined methods using built-in methods such as `.from()`, `.select()`, `.eq()`, and `.ilike()` rather than writing raw SQL. The database is defined across three SQL migration files and five tables: `users`, `pantries`, `pantry_ingredients`, `user_preferences`, and `recipes`.

The `pantries` table stores each pantry with a UUID primary key, a name, and a `last_updated` date that tracks the most recent modification to its contents.

The `users` table references Supabase's built-in `auth.users` table through a foreign key on `user_id`, and stores the user's first name, last name, email, and a reference to their pantry. This link between users and pantries is what allows the app to associate a user with their ingredients.

The `pantry_ingredients` table is the largest table in the schema and tracks each ingredient in a pantry. Each ingredient row stores a Spoonacular ID that ties it back to the Spoonacular API, a normalized name for search and display, a category, a quantity and unit, an optional expiration date, and an image URL.

The `user_preferences` table stores dietary restrictions as a text array and a measurement unit preference, both linked to a user via their id.

The `recipes` table uses a composite primary key of `recipe_id` and `user_id`, which allows different users to independently save or mark the same recipe as made. The `saved` boolean tracks whether the user has saved a recipe, and the `made_on` date records when the user last cooked it.

Security is enforced at the database level through PostgreSQL Row-Level Security (RLS). RLS is enabled on all five tables, and each table has a policy that restricts all operations to only the authenticated user's own data. For the `users`, `user_preferences`, and `recipes` tables, the policy checks that the row's `user_id` matches the currently authenticated user via Supabase's `auth.uid()` function. For the `pantries` and `pantry_ingredients` tables, the policy performs a subquery to verify that the pantry belongs to the authenticated user. This means that even if a client were to attempt to query another user's data directly, the database itself would block the request.

The database also uses two triggers to automate common operations. The first trigger, `on_auth_user_created`, fires after a new row is inserted into Supabase's `auth.users` table. It runs the `handle_new_user()` function, which creates a new pantry, inserts a corresponding row in the `users` table linked to that pantry, and inserts a default row in `user_preferences`. This means that when a user signs up, all of their associated database records are created automatically without the application needing to make multiple calls. The second trigger, `trg_update_pantry_last_updated`, fires after any insert, update, or delete on the `pantry_ingredients` table and updates the parent pantry's `last_updated` date to the current date.

Several aspects of the database were designed to support features that were not fully implemented on the client side due to time constraints. The `pantry_ingredients` table includes the columns `low_stock_threshold` and `in_stock` to power low-stock warnings or filtering, but neither was implemented in the UI. The `user_preferences` table includes `dietary_restrictions` and `calorie_target` fields. Dietary restrictions were implemented in the UI, but currently, they do not write to the database. Calorie target was not implemented at all. The schema design also deliberately supports shared pantries, where multiple users can be assigned to the same pantry through a dedicated service module, but this feature was not deployed in the frontend. These fields and relationships remain in the schema for future development.

Services

The service layer sits between the app's UI components and the database. All service functions interact with Supabase through a typed client. The Supabase client is initialized with the project's URL and anonymous key, and is configured with `AsyncStorage`. This provides session persistence on mobile and automatic token refreshing that pauses when the app is

running in the background and resumes when it regains focus. TypeScript types for every table's insert, update, and row shapes are auto-generated from the database schema, which provides safety at the compilation level across the entire service layer.

The service layer is organized into six TypeScript modules, each responsible for a specific domain.

The pantry service handles creating, reading, renaming, and deleting pantries. The ingredient service handles adding, updating, deleting, and querying ingredients within a pantry, including lookups by name or Spoonacular ID. It also contains an ingredient-image backfill function that fetches standard images from the Spoonacular API for any product that did not automatically receive one.

The recipe service handles saving and retrieving recipes, but also includes the `makeRecipe` function, which is one of the most complex operations in the app. When a user marks a recipe as made, this function fetches the recipe's full ingredient list from the Spoonacular API, matches each recipe ingredient against the user's pantry uniquely using the Spoonacular ID, converts the recipe's required amount into the same unit as the pantry ingredient using a unit conversion utility module, and then deducts the appropriate quantity. Lastly, it upserts the recipe record with the current date as the `made_on` value. Upsert was used because filling the recipe table with every user-recipe pair would be unreasonable, yet recipes need to be both added and updated. Instead, when a new recipe is saved or made, the app inserts a new record. From then on, if the same recipe is made again or instead saved, the app will update the row. For best practice, this prevents race conditions, unlike a read followed by a write.

The preferences service provides functions for reading and updating a user's dietary preferences.

The pantry assignment service is the dedicated module for utilizing the 1:Many pantry-to-user relationship, but as described above, it was not implemented due to time constraints.

The sixth and final module is the API service. The API service handles all communication with the Spoonacular API, including:

1. Barcode lookups
2. Ingredient name normalization
3. Ingredient image searches
4. Recipe searches by ingredient list
5. Recipe ingredients fetching
6. Recipe instruction fetching

The API service also includes a unit conversion and ingredient-matching system that compares a recipe's required ingredients against the user's pantry contents, normalizing between mass and volume units to determine whether the user has enough of each ingredient. All API responses are cached using TanStack Query (React Query) with a 1-hour expiration. Automatic refetching is disabled to minimize redundant network requests, given Spoonacular's rate-limited API.

Project Setup Instructions

This section covers downloading a copy of the Pantry Chef mobile app source code, running it in an Android emulator, and building an Android APK. We'll cover downloading the necessary software, configuring the project to run on Android, and the building process.

Software Setup

You'll need a few software packages installed on your local computer: [GitHub Desktop](#), [Node.js](#), [Visual Studio Code](#), and [Android Studio](#). You'll also need accounts on [Supabase](#) and [Spoonacular](#). Their respective websites have installation and setup instructions. During Node.js setup on Windows, make sure the "Add to PATH" option is enabled so it will be available on the command line later. Once you've installed GitHub Desktop, clone the repository from this link: <https://github.com/tylertgilbert50/pantry-chef>.

Open the resulting folder in Visual Studio Code. Inside VS Code's terminal, change directories to the project root folder and run the following command: `"npm i"`. This command installs the required modules with Node's package manager.

Database and API Setup

To set up the database, log into Supabase. Inside Supabase, click on "New project". Fill out the new project form and click "Create new project". Navigate to the SQL editor in the left panel. To create the project database, copy and paste the contents of each SQL file under "supabase/migrations" into the SQL editor and run the file. Run the files in this order: "create_tables.sql", "rls_policies.sql", "functions_triggers_views.sql". You may be prompted to enable RLS before running, which is correct, and you should enable it.

To set up Spoonacular, log in to Spoonacular and navigate to "Profile & API Key" in the left panel. Click on "Show/Hide API Key" to reveal the API key.

To configure the project to use Supabase and Spoonacular, navigate to the file named "app.json" in the project root folder. Here you'll enter your Supabase instance URL, Supabase key, and Spoonacular API key. The key names are "EXPO_PUBLIC_SUPABSE_URL", "EXPO_PUBLIC_SUPABSE_ANON_KEY", and "EXPO_PUBLIC_SPOONACULAR_API_KEY". Change the values of these keys to match your Supabase and Spoonacular information.

Running on an emulator

Open Android Studio and select "More Actions", then "Virtual Device Manager". This will open the virtual device manager, where you can install and launch virtual Android devices. You can create a new device by clicking the "+" button. We recommend the Pixel 9 for testing Pantry Chef, but other devices are also compatible. We do not recommend choosing Android 17, as we found it unstable in the emulator (screen flickering and freezing). Instead, we recommend using Android 16. Once you have created your virtual device, you can launch it by clicking the

play button on the right side of the window. The device may take a few minutes to start for the first time. Once it's finished starting, we're ready to test the app. Inside Visual Studio Code's terminal, run the following command: `"npx expo"`. Expo may take a few seconds to bundle the code. It should display a QR code and address once it's ready. Press `"a"` to open the app in Expo Go inside the Android emulator. Expo should automatically recognize the emulator and install Expo Go if it isn't already installed. Changes to the codebase will be synced to the app in real time. If the app fails to sync changes, refresh the session by pressing `"r"` in the terminal. When you're finished testing, you can exit Expo by pressing `control+c`.

Running on a physical device

During testing, you may wish to use a physical device, such as an Android phone. To do this, install the Expo Go app from the Google Play Store. Inside Visual Studio Code's terminal, run the following command: `"npx expo"`. This will create a QR code. Open the Expo Go app and tap `"Scan QR code"`. Scan the QR code from the Expo terminal. The app should open on the testing device. If it doesn't, see the troubleshooting section below. Both devices need to be connected to the same wifi network.

To test on an Apple device, such as an iPhone, install the Expo Go app from the App Store. Start Expo from the Visual Studio Code terminal using `"npx expo"`. Open the camera app on the iPhone and scan the QR code created by Expo. The Expo Go app should open and run the project.

Building an Android APK

To build an Android APK, we'll use EAS build services. To confirm that the project is ready to build, install expo-doctor using `"npm install expo-doctor"` and run `"npx expo-doctor"`. This will report any errors found with the project before committing to building on EAS servers. Once you're ready to build, create an account at expo.dev. Then, install EAS CLI by running `"npm install -g eas-cli"`. Log in to EAS by running `"eas login"`. Next, we'll configure the project for building by running `"eas build:configure"`. Finally, build the project using `"eas build -p android --profile preview"`. You can monitor the build status using the link EAS provides in the terminal. It may take several hours if you're using the free tier. Once the build finishes, EAS will provide a link to download the APK and a QR code that links to it. If you have a development phone, scanning the QR code is the quickest way to test your build. We found the `"launch in emulator"` option unreliable and recommend testing on a physical device.

Troubleshooting

Here is a list of common issues you may encounter:

- Name clash error: Visual Studio Code may create duplicate folders or files with the same name as the original, but with `"Name Clash"` appended to the name. You may delete these files.
- Unresponsive app: If the app becomes unresponsive in the Android emulator, close the Expo Go app, exit the Expo session in VS Code, start a new Expo session, and press `"a"` to open the app on the Android emulator.

- Expo Go is unable to connect from a physical device: Make sure your Expo server device and Android device are on the same wifi network. If it still won't connect, start Expo using tunnel mode: `"npx expo --tunnel"`. We found that this fix resolves most connection errors.
- Various errors with the app's state not reflecting new changes: start Expo using `"npx expo --clear"`
- Node modules acting weird: Delete the `node_modules` folder and run `"npm i"` again.

Known Issues

Pantry Chef has several known issues, including

- Client-side API access: This is likely the project's biggest issue. A malicious user could extract our Spoonacular API key and use it outside of Pantry Chef. This issue could be fixed by creating our own backend to proxy API requests.
- Non-functional password reset system: Password reset emails never arrive, and we haven't completed the necessary troubleshooting to identify the cause.
- Non-functional allergy/dietary preferences: The backend currently doesn't support filtering by allergies or dietary preferences. The UI component is complete, but we ran out of time to fully implement this feature.
- Recipe matching: Spoonacular API doesn't predictably return recipes based on available ingredients. For example, a pantry stocked with waffle ingredients will initially display the waffle recipe. Upon adding a non-waffle ingredient (like spinach), the waffle recipe will move back into the returned search results, leaving no matching recipes and requiring Pantry Chef to request more recipes to find it. To resolve this issue, we would likely need to switch data sources away from Spoonacular.
- Barcode scanner sensitivity: The barcode scanner will scan the same barcode over and over without a limit. This results in the form popping up again instantly when the user closes it. This is partially user error and partially a logic error. The user is pointing the camera at a barcode, but they likely don't intend to scan the same barcode instantly when they just scanned it.